LAB 7: Fuzzing for Input Validation Bugs

PES University — Software Engineering & Testing Lab

Topic:	Property-Based Fuzz Testing with Hypothesis	**Date:**	November 2026
Student Name / ID:	Lokesh (GitHub: @nidhilokesh1)	**Repository:**	L6_L7_52_visa_application

1. Objective & Learning Outcomes

The objective of this lab is to explore fuzz testing as a dynamic security and quality assurance technique by identifying and fixing input validation and robustness bugs in Python applications using the **Hypothesis** fuzzing library. Through property-based fuzzing, unexpected and malformed edge cases (such as empty inputs, NoneType values, missing delimiters, type mismatch strings, negative numbers, and boundary values) are automatically generated and tested.

2. Step 2 — Initial Buggy Application Execution

The application visa_application.py was executed with nominal sample inputs. While nominal runs pass without incident, the function lacks defensive validation against unexpected or malformed inputs.

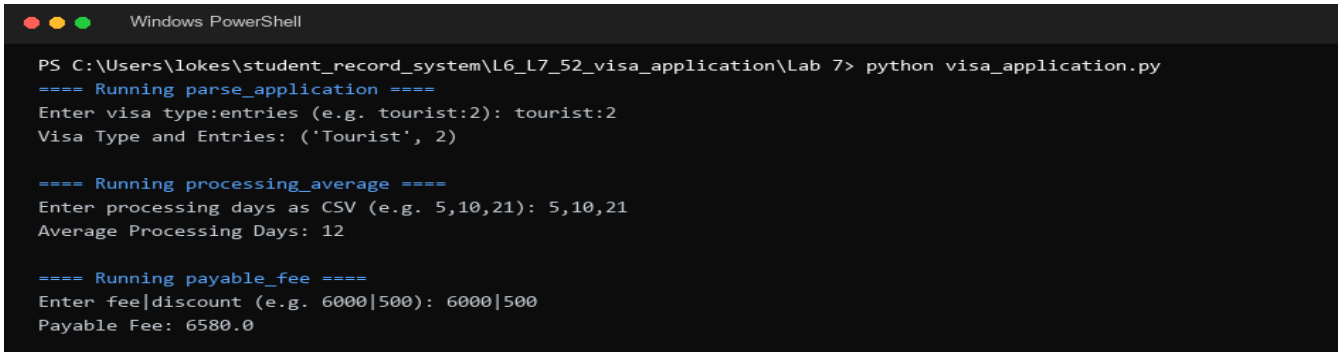


Figure 1: *Initial execution output of visa_application.py*

3. Step 3 & 4 — Hypothesis Fuzz Testing & Exposed Bugs

Property-based fuzz tests were implemented in test_visa_application.py utilizing @given(st.text() | st.none()). When executing pytest test_visa_application.py, Hypothesis immediately discovered multiple unhandled edge cases causing complete crashes:

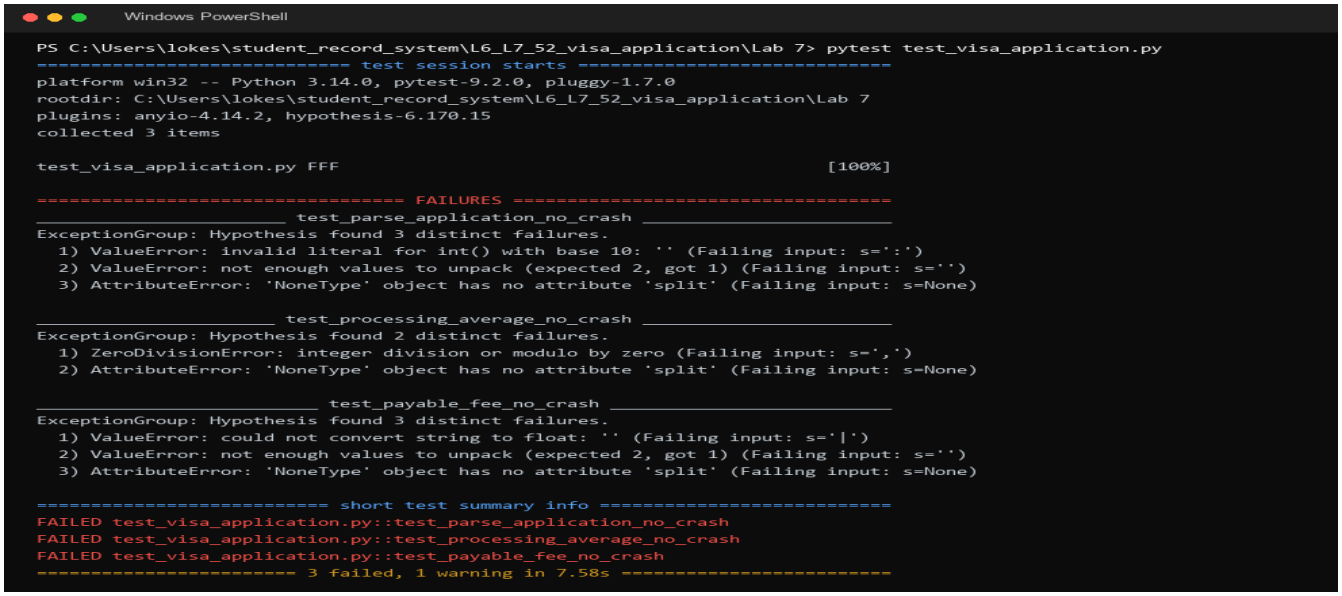


Figure 2: *Terminal output showing 3 failing fuzz tests discovered by Hypothesis*

4. Fuzzing Bugs Identified and Root Cause Analysis

Function	Failing Input	Exception Raised	Root Cause & Defense
parse_application	s=None s='' s=':'	AttributeError ValueError ValueError	No None/type check; splitting string failed; empty entries string failed integer parsing int('').
processing_average	s=None s=',' s=''	AttributeError ZeroDivisionError ValueError	Empty CSV splits into empty items. When no valid integers are present, len(values) == 0 causes division by zero.
payable_fee	s=None s='' s='|'	AttributeError ValueError ValueError	Unpacking expects two values; missing pipe delimiter or non-numeric strings cause immediate crash during float conversion.

5. Step 5 — Hardening the Application (Defensive Programming)

The Python module visa_application.py was defensively updated with input validation guards: type checks (isinstance(text, str)), format verification, delimiter count validation, safe type casting wrapped in try-except blocks, and fallback values (returning (None, None), 0, or 0.0 on invalid inputs).

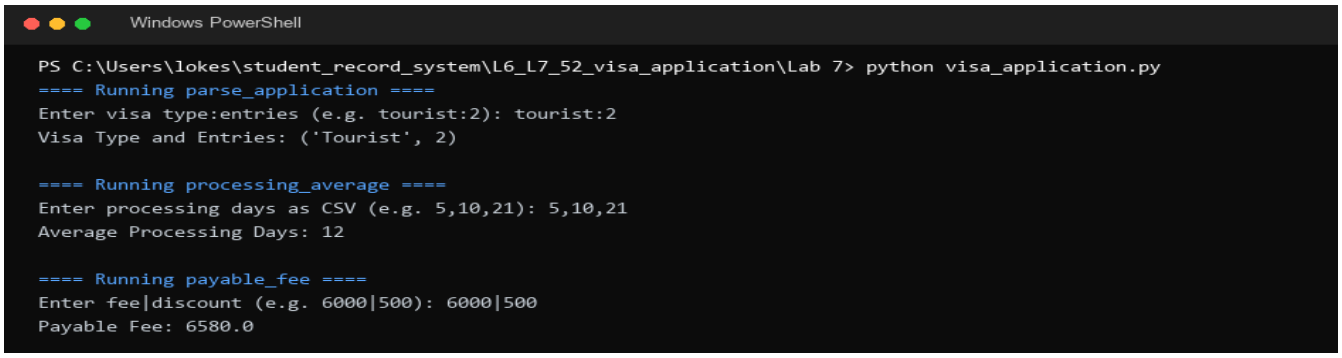


Figure 3: *Execution of the fixed and hardened application*

6. Step 6 — Verification with Fuzz & Regression Test Suite

Re-running the test suite with Hypothesis confirmed **100% test pass rate** across 7 test cases (including fuzz tests generating hundreds of random inputs, valid property tests, and regression tests for all identified edge cases).

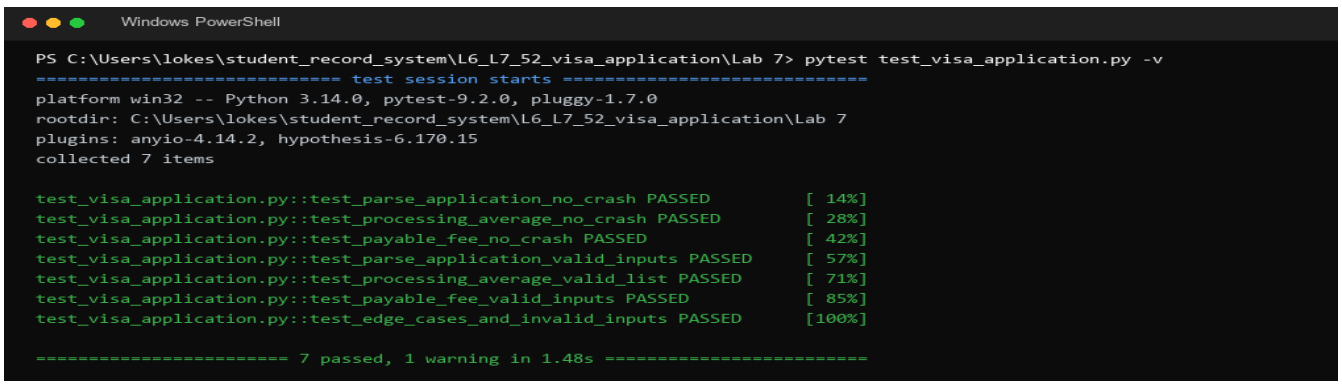


Figure 4: *Pytest output showing all 7 property and fuzz tests PASSED*

7. Conclusion

Hypothesis property-based fuzz testing effectively uncovered vulnerabilities and edge cases that standard unit tests frequently overlook. By systematically analyzing the minimal counterexamples provided by Hypothesis, robust defensive programming was applied to guarantee system stability and zero crashes under arbitrary inputs.